

```
# Install (run once in Colab/Jupyter)
!pip install transformers
!pip install torch

# Import libraries
from transformers import pipeline
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transform
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transf
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from tr
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transf
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transfo
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transfo
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from h
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from h
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from http
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.2
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdow
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torc
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torc
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torc
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from ji
```

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# Custom class to replicate pipeline behavior
class CustomTranslator:
    def __init__(self, tokenizer, model):
        self.tokenizer = tokenizer
        self.model = model

    def __call__(self, text):
        inputs = self.tokenizer(text, return_tensors="pt", padding=True, truncation=True)
        outputs = self.model.generate(**inputs)
        translated_text = self.tokenizer.decode(outputs[0], skip_special_tokens=True)
        return [{'translation_text': translated_text}]

# English → French
tokenizer_fr = AutoTokenizer.from_pretrained("Helsinki-NLP/opus-mt-en-fr")
model_fr = AutoModelForSeq2SeqLM.from_pretrained("Helsinki-NLP/opus-mt-en-fr")
translator_fr = CustomTranslator(tokenizer_fr, model_fr)

# English → German
tokenizer_de = AutoTokenizer.from_pretrained("Helsinki-NLP/opus-mt-en-de")
```

```
model_de = AutoModelForSeq2SeqLM.from_pretrained("Helsinki-NLP/opus-mt-en-de")
translator_de = CustomTranslator(tokenizer_de, model_de)
```

```
# English → Hindi
tokenizer_hi = AutoTokenizer.from_pretrained("Helsinki-NLP/opus-mt-en-hi")
model_hi = AutoModelForSeq2SeqLM.from_pretrained("Helsinki-NLP/opus-mt-en-hi")
translator_hi = CustomTranslator(tokenizer_hi, model_hi)
```

```
tokenizer_config.json: 100% 42.0/42.0 [00:00<00:00, 3.35kB/s]
source.spm: 100% 778k/778k [00:00<00:00, 11.5MB/s]
target.spm: 100% 802k/802k [00:00<00:00, 41.7MB/s]
vocab.json: 1.34M/? [00:00<00:00, 57.3MB/s]
/usr/local/lib/python3.12/dist-packages/transformers/models/arian/tokenization_arian.py:176: U
warnings.warn("Recommended: pip install sacremoses.")
pytorch_model.bin: 100% 301M/301M [00:03<00:00, 174MB/s]
Loading weights: 100% 258/258 [00:00<00:00, 914.03it/s, Materializing param=model.shared.weight]
model.safetensors: 100% 301M/301M [00:05<00:00, 1.50GB/s]
The tied weights mapping and config for this model specifies to tie model.shared.weight to model
The tied weights mapping and config for this model specifies to tie model.shared.weight to model
generation_config.json: 100% 293/293 [00:00<00:00, 6.75kB/s]
config.json: 1.33k/? [00:00<00:00, 17.8kB/s]
tokenizer_config.json: 100% 42.0/42.0 [00:00<00:00, 3.70kB/s]
source.spm: 100% 768k/768k [00:00<00:00, 1.39MB/s]
target.spm: 100% 797k/797k [00:00<00:00, 1.06MB/s]
vocab.json: 1.27M/? [00:00<00:00, 40.1MB/s]
pytorch_model.bin: 100% 298M/298M [00:02<00:00, 309MB/s]
Loading weights: 100% 258/258 [00:00<00:00, 865.65it/s, Materializing param=model.shared.weight]
model.safetensors: 100% 298M/298M [00:02<00:00, 201MB/s]
The tied weights mapping and config for this model specifies to tie model.shared.weight to model
The tied weights mapping and config for this model specifies to tie model.shared.weight to model
generation_config.json: 100% 293/293 [00:00<00:00, 25.0kB/s]
config.json: 1.39k/? [00:00<00:00, 131kB/s]
tokenizer_config.json: 100% 44.0/44.0 [00:00<00:00, 5.71kB/s]
source.spm: 100% 812k/812k [00:00<00:00, 11.8MB/s]
target.spm: 100% 1.07M/1.07M [00:00<00:00, 12.3MB/s]
vocab.json: 2.10M/? [00:00<00:00, 27.8MB/s]
pytorch_model.bin: 100% 306M/306M [00:02<00:00, 188MB/s]
Loading weights: 100% 258/258 [00:00<00:00, 811.96it/s, Materializing param=model.shared.weight]
model.safetensors: 0% 0.00/306M [00:00<?, ?B/s]
The tied weights mapping and config for this model specifies to tie model.shared.weight to model
The tied weights mapping and config for this model specifies to tie model.shared.weight to model
generation_config.json: 100% 293/293 [00:00<00:00, 10.1kB/s]
```

```
sentences = [
    "I love learning artificial intelligence.",
    "The weather is very pleasant today.",
    "She completed her project successfully.",
    "What are you doing right now?",
```

```

"Machine learning is transforming industries.",
"He went to the market and bought fruits.",
"If you study well, you will pass the exam.",
"Can you help me with this problem?",
"The meeting was postponed due to rain.",
"Artificial intelligence is a rapidly growing field.",
"Let's go for a walk in the evening.",
"The system failed because of a technical error.",
"Do you like watching movies?",
"Although it was raining, we continued the journey.",
>Data science is used in healthcare and finance."
]

```

```

print("---- English to French ----\n")

for sentence in sentences:
    result = translator_fr(sentence)
    print("EN:", sentence)
    print("FR:", result[0]['translation_text'])
    print()

```

```
---- English to French ----
```

```

EN: I love learning artificial intelligence.
FR: J'adore apprendre l'intelligence artificielle.

```

```

EN: The weather is very pleasant today.
FR: Le temps est très agréable aujourd'hui.

```

```

EN: She completed her project successfully.
FR: Elle a terminé son projet avec succès.

```

```

EN: What are you doing right now?
FR: Qu'est-ce que tu fais en ce moment ?

```

```

EN: Machine learning is transforming industries.
FR: L'apprentissage automatique transforme les industries.

```

```

EN: He went to the market and bought fruits.
FR: Il est allé au marché et a acheté des fruits.

```

```

EN: If you study well, you will pass the exam.
FR: Si vous étudiez bien, vous réussirez l'examen.

```

```

EN: Can you help me with this problem?
FR: Pouvez-vous m'aider avec ce problème ?

```

```

EN: The meeting was postponed due to rain.
FR: La réunion a été reportée en raison de la pluie.

```

```

EN: Artificial intelligence is a rapidly growing field.
FR: L'intelligence artificielle est un domaine en croissance rapide.

```

```

EN: Let's go for a walk in the evening.
FR: Allons faire une promenade le soir.

```

```

EN: The system failed because of a technical error.
FR: Le système a échoué en raison d'une erreur technique.

```

```

EN: Do you like watching movies?
FR: Tu aimes regarder des films ?

```

```

EN: Although it was raining, we continued the journey.
FR: Même s'il pleuvait, nous avons poursuivi le voyage.

```

```

EN: Data science is used in healthcare and finance.
FR: La science des données est utilisée dans les soins de santé et les finances.

```

```
for sentence in sentences:
```

```
tr = translator_tr(sentence)[0]['translation_text']
de = translator_de(sentence)[0]['translation_text']
hi = translator_hi(sentence)[0]['translation_text']

print("EN:", sentence)
print("FR:", fr)
print("DE:", de)
print("HI:", hi)
print("-" * 50)
```

DE: Was machst du gerade?

HI: तुम अभी क्या कर रहे हो?

EN: Machine learning is transforming industries.

FR: L'apprentissage automatique transforme les industries.

DE: Maschinelles Lernen verändert die Industrie.

HI: मशीन पढ़ना उद्योगों को परिवर्तित कर रहा है.

EN: He went to the market and bought fruits.

FR: Il est allé au marché et a acheté des fruits.

DE: Er ging auf den Markt und kaufte Früchte.

HI: वह बाजार में गया और फल लाया.

EN: If you study well, you will pass the exam.

FR: Si vous étudiez bien, vous réussirez l'examen.

DE: Wenn du gut lernst, wirst du die Prüfung bestehen.

HI: अगर आप अच्छी तरह से अध्ययन करें, तो आप परीक्षा को पार करेंगे ।

EN: Can you help me with this problem?

FR: Pouvez-vous m'aider avec ce problème ?

DE: Können Sie mir bei diesem Problem helfen?

HI: आप इस समस्या के साथ मेरी मदद कर सकते हैं?

EN: The meeting was postponed due to rain.

FR: La réunion a été reportée en raison de la pluie.

DE: Das Treffen wurde wegen Regens verschoben.

HI: इस सभा को बारिश के लिए देर हो चुकी थी ।

EN: Artificial intelligence is a rapidly growing field.

FR: L'intelligence artificielle est un domaine en croissance rapide.

DE: Künstliche Intelligenz ist ein schnell wachsendes Feld.

HI: कलावादी बुद्धि तेज़ी से बढ़ती हुई क्षेत्र है ।

EN: Let's go for a walk in the evening.

FR: Allons faire une promenade le soir.

DE: Gehen wir abends spazieren.

HI: चलो शाम को सैर के लिए चलते हैं.

EN: The system failed because of a technical error.

FR: Le système a échoué en raison d'une erreur technique.

DE: Das System scheiterte aufgrund eines technischen Fehlers.

HI: तकनीकी त्रुटि के कारण तंत्र असफल.

EN: Do you like watching movies?

FR: Tu aimes regarder des films ?

DE: Sehen Sie sich gern Filme an?

HI: क्या आप फिल्मों को देखना पसंद करते हैं?

EN: Although it was raining, we continued the journey.

FR: Même s'il pleuvait, nous avons poursuivi le voyage.

DE: Obwohl es regnete, setzten wir die Reise fort.

HI: हालांकि बारिश हो रही थी, फिर भी हमने सफर जारी रखा ।

EN: Data science is used in healthcare and finance.

FR: La science des données est utilisée dans les soins de santé et les finances.

DE: Data Science wird in der Gesundheits- und Finanzbranche eingesetzt.

HI: सेहत के मामले में डेटा विज्ञान का इस्तेमाल किया जाता है और खर्चा भी ।

```
complex_sentences = [
```

```
"Although the weather was bad, we decided to continue the journey.",
```

```
"If artificial intelligence continues to grow, it will change the future.",
```

```
"The project which was started last year has now been completed "
```

```
the project, which was started last year, has now been completed. ",  
"Can you explain how machine learning works in simple terms?",  
"Despite facing many challenges, she achieved her goals."  
]
```

```
for sentence in complex_sentences:  
    print("EN:", sentence)  
    print("FR:", translator_fr(sentence)[0]['translation_text'])  
    print()
```

EN: Although the weather was bad, we decided to continue the journey.
FR: Bien que le temps ait été mauvais, nous avons décidé de poursuivre le voyage.

EN: If artificial intelligence continues to grow, it will change the future.
FR: Si l'intelligence artificielle continue de croître, elle changera l'avenir.

EN: The project, which was started last year, has now been completed.
FR: Le projet, qui a débuté l'année dernière, est maintenant achevé.

EN: Can you explain how machine learning works in simple terms?
FR: Pouvez-vous expliquer comment fonctionne l'apprentissage automatique en termes simples?

EN: Despite facing many challenges, she achieved her goals.
FR: Malgré de nombreux défis, elle a atteint ses objectifs.

Start coding or [generate](#) with AI.